RTSP实时图传结合AIDemo

RTSP实时图传结合AIDemo
- 效果演示
- 示例代码
- 修改步骤
  - 1. 准备工作
    - 1.1 配置WIFI设置
  - 2. 修改_do_rtsp_stream()
  - 3. 复制FaceLandMark以及相关的类
  - 4. 修改FaceLandMark以及相关类
- 注意事项【必看】

效果演示

> 注：K230上的WIFI芯片性能有限，请在靠近WIFI信号热点的地方使用图传

> *本例程为实验性内容，在不同网络环境下有概率出现无法高质量运行的可能性

我们运行这节修改后的例程，在控制台会有类似如下的输出信息：

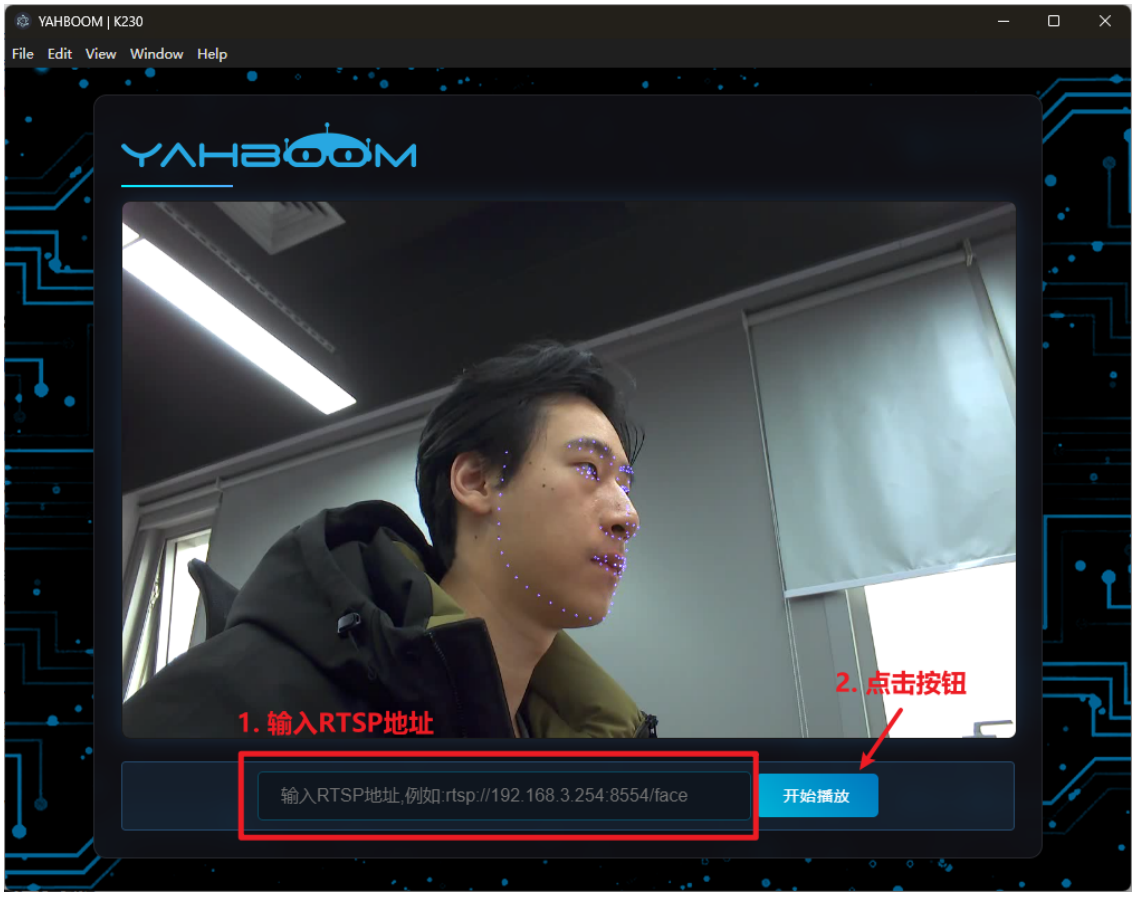
```
[WIFI] 连接网络中 ...
True
[WIFI] 连接网络成功
[RTSP] 启动中 ..
find sensor gc2093_csi2, type 30, output 1920x1080@60
buffer pool :  4
sensor(0), mode 0, buffer_num 4, buffer_size 0
[RTSP] 启动成功, 地址: rtsp://192.168.3.254:8554/video
```

这个地址就是RTSP推流的地址。

我们可以用支持RTSP流的播放器来远程播放K230的画面，本节以人脸关键点检测为例

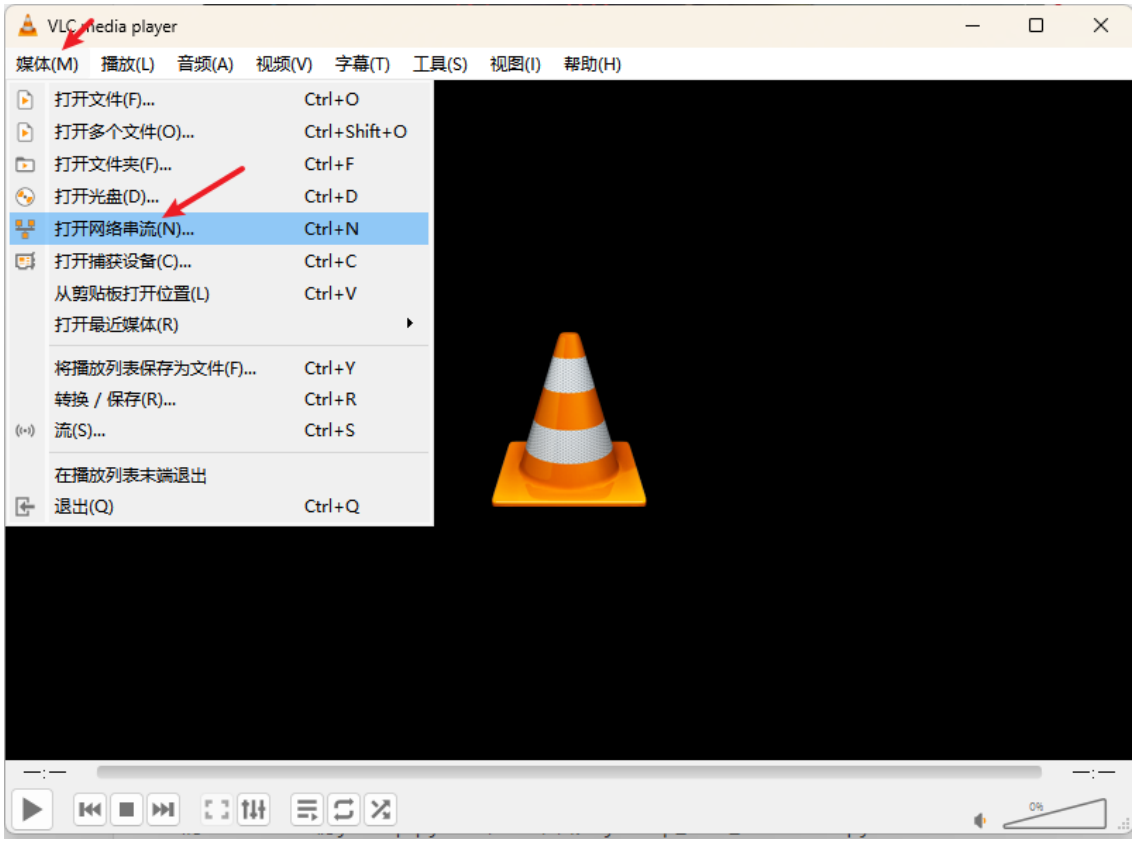
可以使用VLC Media Player或者我们提供的rtsp_player，这两个软件都在我们的附件【K230 教程\0.开发板资料\开发软件】中了

1. 推荐优先使用rtsp_player，功能简洁，不需要繁琐的配置，双击打开即可使用



2. VLC Media Player是一个多功能的强大视频播放器，但是设置较为繁琐

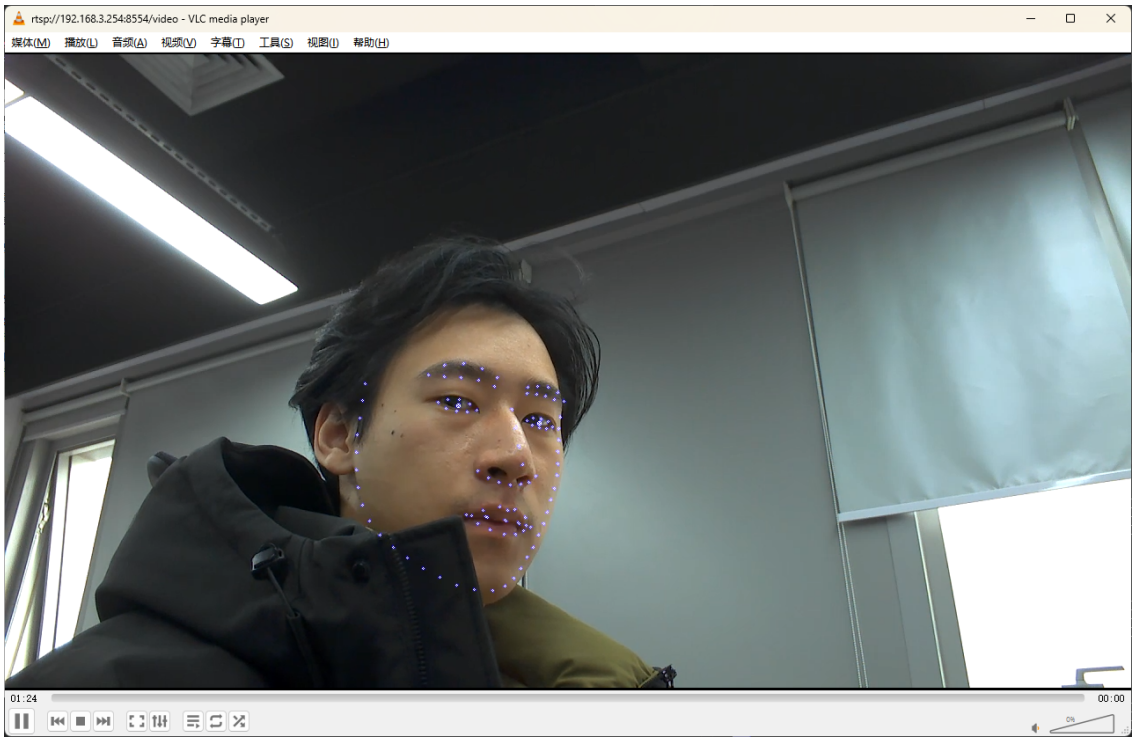
我们打开VLC Media Player，点击“媒体”->"打开网络串流"



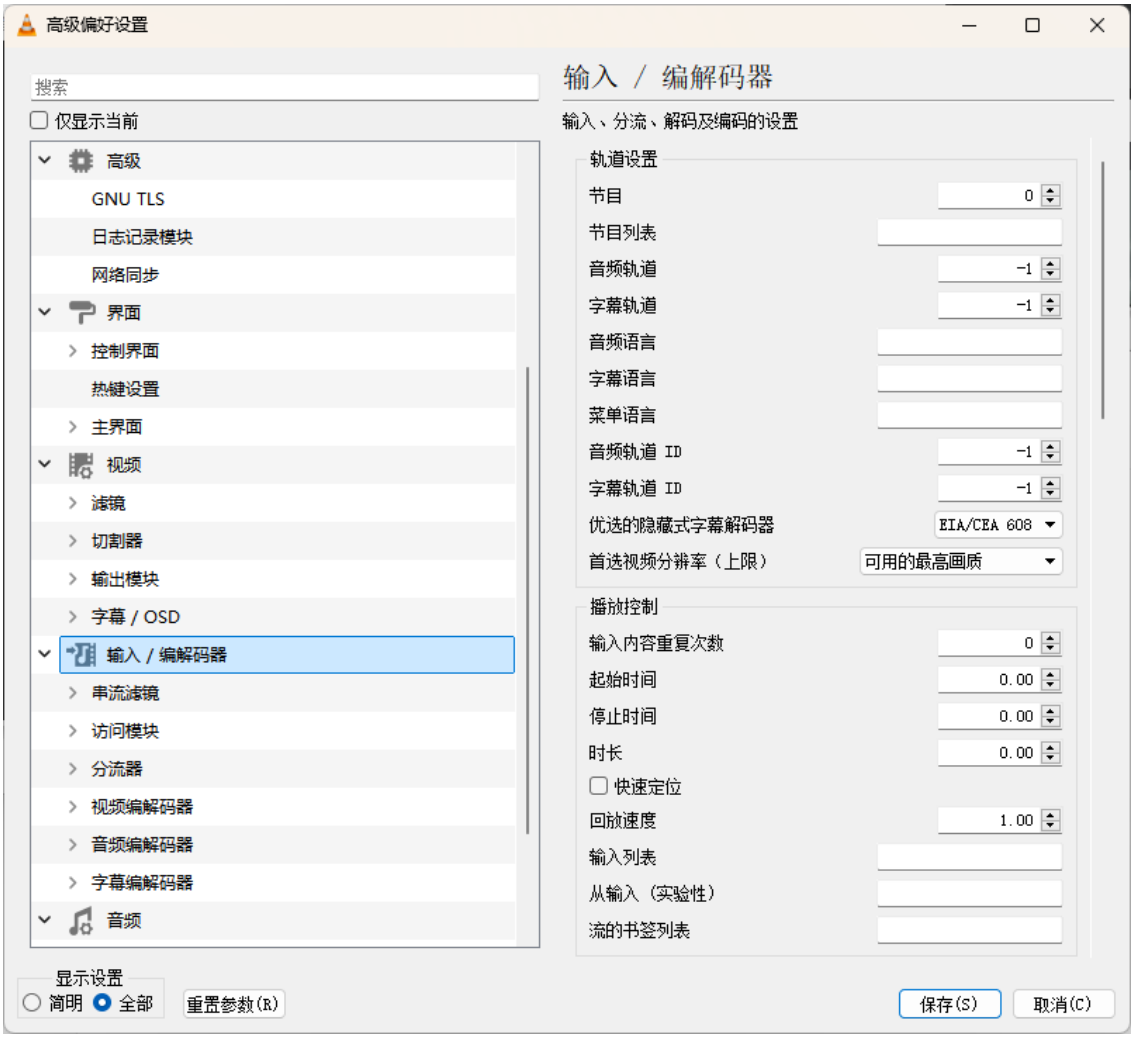
输入url地址，在“显示更多选项”这里把延迟改低一点，10~100ms左右



修改完成后点击右下角【播放】按钮，即可远程播放K230传过来的视频



如果播放性能不佳，一个是确保网络速度以及连接的稳定性，另一个是可以在“偏好设置”里面尝试优化



示例代码

我们这里有一个已经修改好的基于人脸检测的图传代码

放在【源码汇总 / 11.Network / 07.rtsp / rtsp_face_detect.py】

修改WIFI参数后运行即可查看效果

```
if __name__ == "__main__":
    print("连接网络中 ...")
    Connect_WIFI("test", "password")
    print("启动中 ..")
```

修改步骤

1. 准备工作

本节教程我们来介绍一下如何实现将 AIDemo 识别后的图像，通过RTSP远程传输

> *本节教程对于初学者来说有一定的挑战性，需要您具备一定的代码基础以及编程能力*

我们以face_landmark（人脸关键点检测）为例,

需要的代码有：

1. empty_rtsp_demo.py【位于 源码汇总 / 11.Network / 07.rtsp / empty_rtsp_demo.py】
2. face_landmark.py【位于 源码汇总 / 07.Face / 02.face_landmark.py】

我们首先复制一份empty_rtsp_demo.py文件，重命名为 empty_rtsp_demo_face_landmark.py

1.1 配置WIFI设置

我们滑到代码最底下，找到 if name == "main"部分

修改Connect_WIFI()方法的参数，改为我们自己的WIFI名称以及密码（最好是纯英文）

```
if __name__ == "__main__":
    print("[WIFI] 连接网络中 ...")

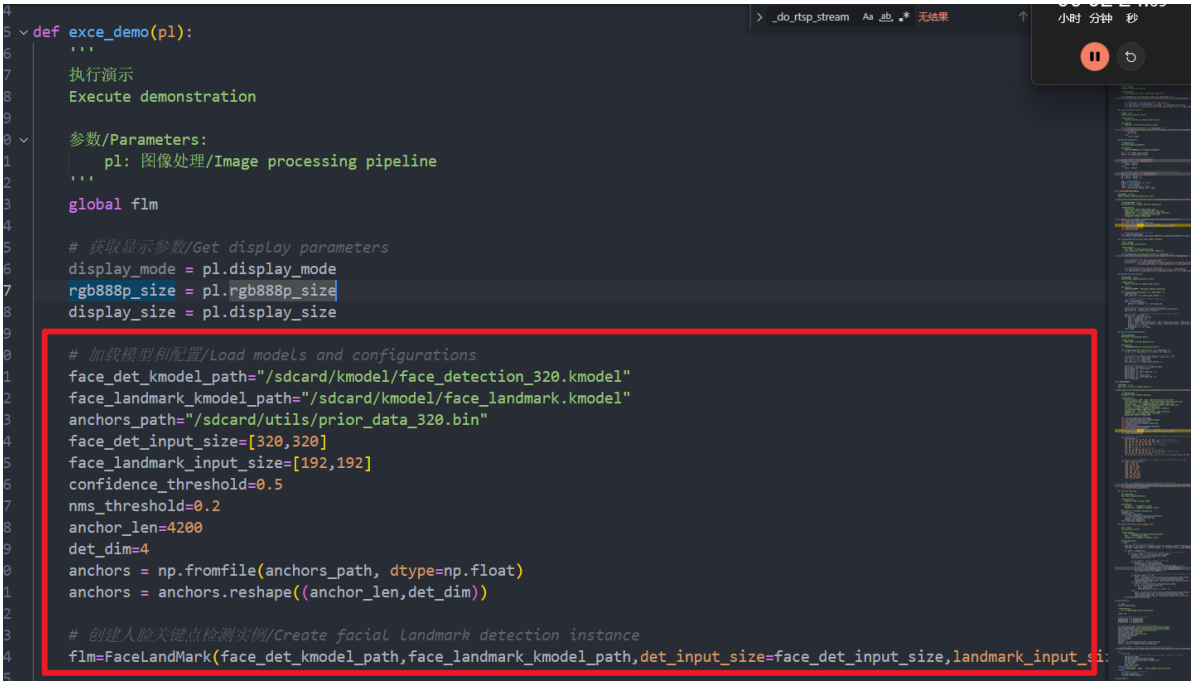
    ####### 修改为自己的WIFI ################
    Connect_WIFI("SSID","PASSWORD")
    ########################################

    print("[WIFI] 连接网络成功")
    print("[RTSP] 启动中 ..")
    time.sleep(1)

   ############## 下面的部分不需要修改 #############
...
```

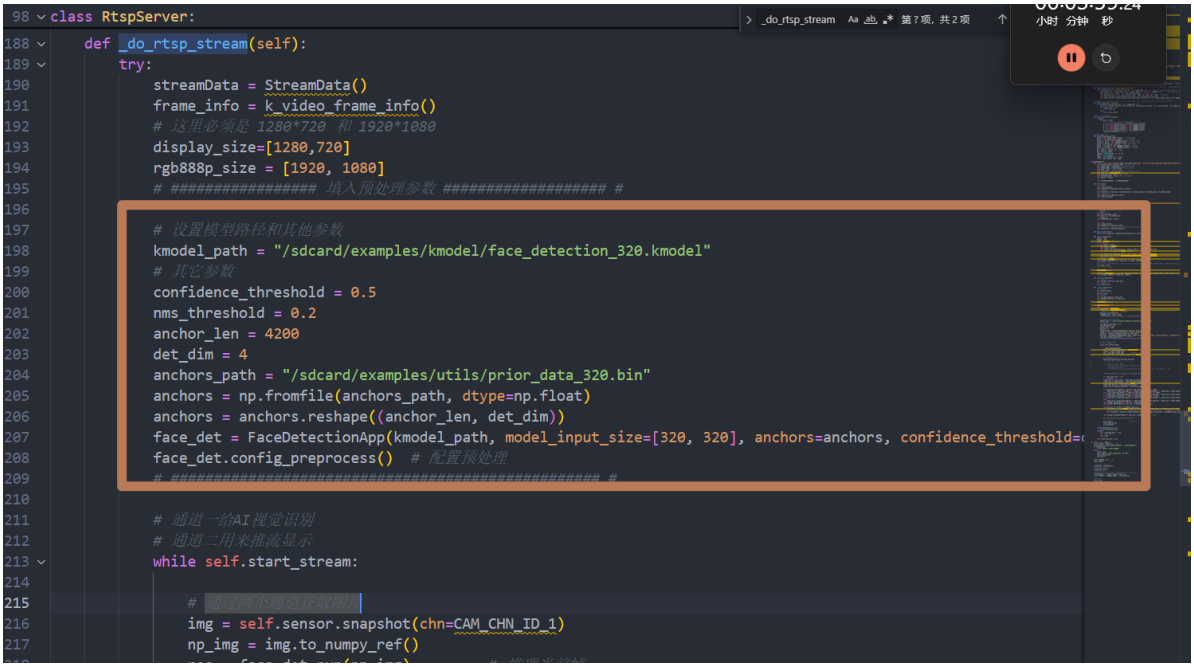
2. 修改_do_rtsp_stream()

复制后我们打开 face_landmark.py ，找到 exce_demo()方法，复制红框中的部分（try-while之前的所有代码）



打开我们刚刚新建的【empty_rtsp_demo_face_landmark.py】文件，找到【_do_rtsp_stream()】方法

用我们复制的红框中的内容替换下面棕色框中的内容



```
class RtspServer:
    def _do_rtsp_stream(self):
        try:
            streamData = StreamData()
            frame_info = k_video_frame_info()
            # 这里必须是 1280*720 和 1920*1080
            display_size=[1280,720]
            rgb888p_size = [1920, 1080]
            # ################ 填入预处理参数 ################## #

            # 设置模型路径和其他参数
            kmodel_path = "/sdcard/examples/kmodel/face_detection_320.kmodel"
            # 其它参数
            confidence_threshold = 0.5
            nms_threshold = 0.2
            anchor_len = 4200
            det_dim = 4
            anchors_path = "/sdcard/examples/utils/prior_data_320.bin"
            anchors = np.fromfile(anchors_path, dtype=np.float)
            anchors = anchors.reshape((anchor_len, det_dim))
            face_det = FaceDetectionApp(kmodel_path, model_input_size=[320, 320], anchors=anchors, confidence_threshold=
            face_det.config_preprocess()  # 配置预处理
            # ################################################ #

            # 通道一给AI视觉识别
            # 通道二用来推流显示
            while self.start_stream:

                # 通过两个通道获取图片
                img = self.sensor.snapshot(chn=CAM_CHN_ID_1)
                np_img = img.to_numpy_ref()
```

替换后代码如下



```
    def _do_rtsp_stream(self):
        try:
            streamData = StreamData()
            frame_info = k_video_frame_info()
            # 这里必须是 1280*720 和 1920*1080
            display_size=[1280,720]
            rgb888p_size = [1920, 1080]


            # 加载模型和配置/Load models and configurations
            face_det_kmodel_path="/sdcard/kmodel/face_detection_320.kmodel"
            face_landmark_kmodel_path="/sdcard/kmodel/face_landmark.kmodel"
            anchors_path="/sdcard/utils/prior_data_320.bin"
            face_det_input_size=[320,320]
            face_landmark_input_size=[192,192]
            confidence_threshold=0.5
            nms_threshold=0.2
            anchor_len=4200
            det_dim=4
            anchors = np.fromfile(anchors_path, dtype=np.float)
            anchors = anchors.reshape((anchor_len,det_dim))

            # 创建人脸关键点检测实例/Create facial landmark detection instance
            flm=FaceLandMark(face_det_kmodel_path,face_landmark_kmodel_path,det_input_size=face_det_input_size,landmark_

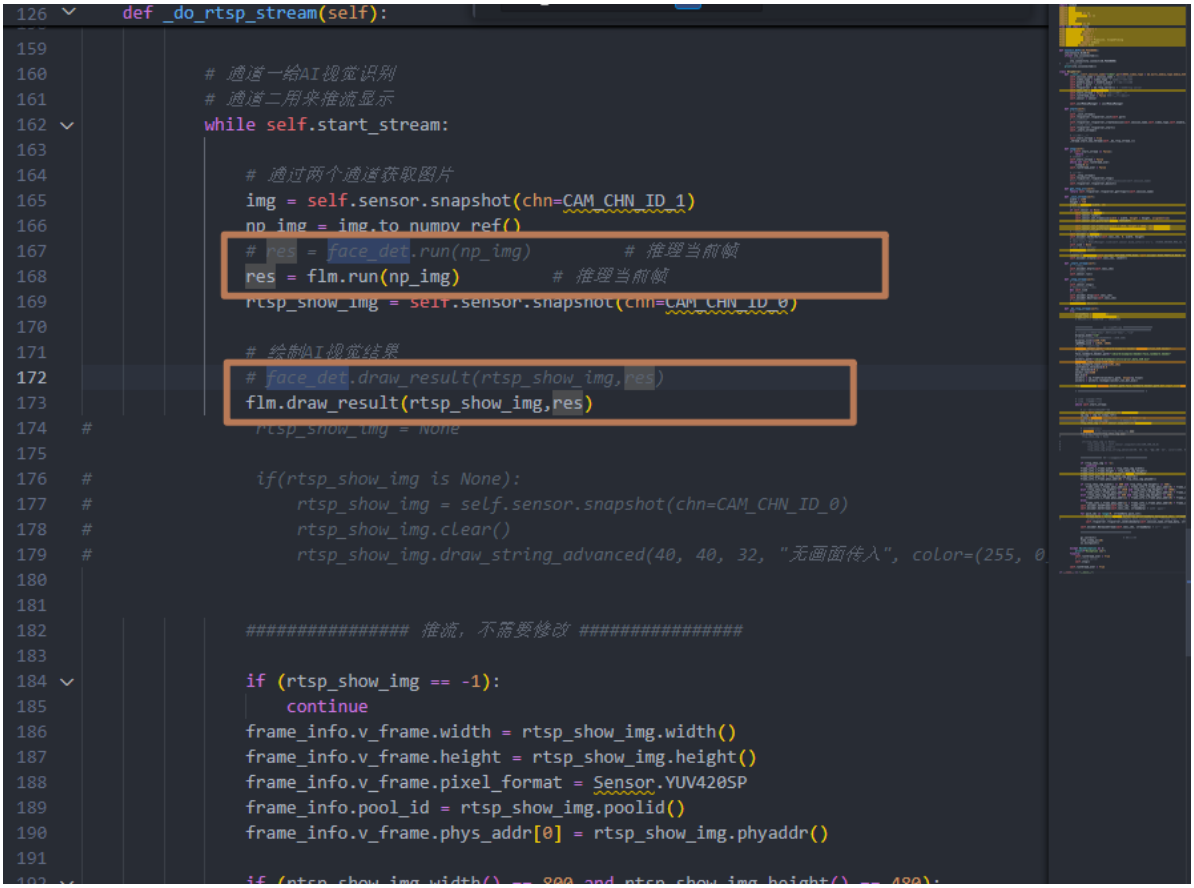
            # ################################################ #

            # 通道一给AI视觉识别
            # 通道二用来推流显示
            while self.start_stream:

                # 通过两个通道获取图片
```

紧接着下面的代码也需要修改，我们把【face_det】改为【flm】(图中被注释掉的部分是修改之前的)

（这里要参照exce_demo()修改，这里这样改是初步的修改，后面还会再进一步修改一下）



```
126     def _do_rtsp_stream(self):
159
160         # 通道一给AI视觉识别
161         # 通道二用来推流显示
162         while self.start_stream:
163
164             # 通过两个通道获取图片
165             img = self.sensor.snapshot(chn=CAM_CHN_ID_1)
166             np_img = img.to_numpy_ref()
167             # res = face_det.run(np_img)          # 推理当前帧
168             res = flm.run(np_img)          # 推理当前帧
169             rtsp_show_img = self.sensor.snapshot(chn=CAM_CHN_ID_0)
170
171             # 绘制AI视觉结果
172             # face_det.draw_result(rtsp_show_img,res)
173             flm.draw_result(rtsp_show_img,res)
174 #           rtsp_show_img = None
175
176 #           if(rtsp_show_img is None):
177 #               rtsp_show_img = self.sensor.snapshot(chn=CAM_CHN_ID_0)
178 #               rtsp_show_img.clear()
179 #               rtsp_show_img.draw_string_advanced(40, 40, 32, "无画面传入", color=(255, 0
180
181
182             ################ 推流，不需要修改 ################
183
184             if (rtsp_show_img == -1):
185                 continue
186             frame_info.v_frame.width = rtsp_show_img.width()
187             frame_info.v_frame.height = rtsp_show_img.height()
188             frame_info.v_frame.pixel_format = Sensor.YUV420SP
189             frame_info.pool_id = rtsp_show_img.poolid()
190             frame_info.v_frame.phys_addr[0] = rtsp_show_img.phyaddr()
191
192             if (rtsp_show_img.width() == 800 and rtsp_show_img.height() == 480):
```

至此，【_do_rtsp_stream()】部分的初步修改就结束了

3. 复制FaceLandMark以及相关的类

本节中我们用到了三个类来实现AI视觉效果，所以三个类我们都要复制

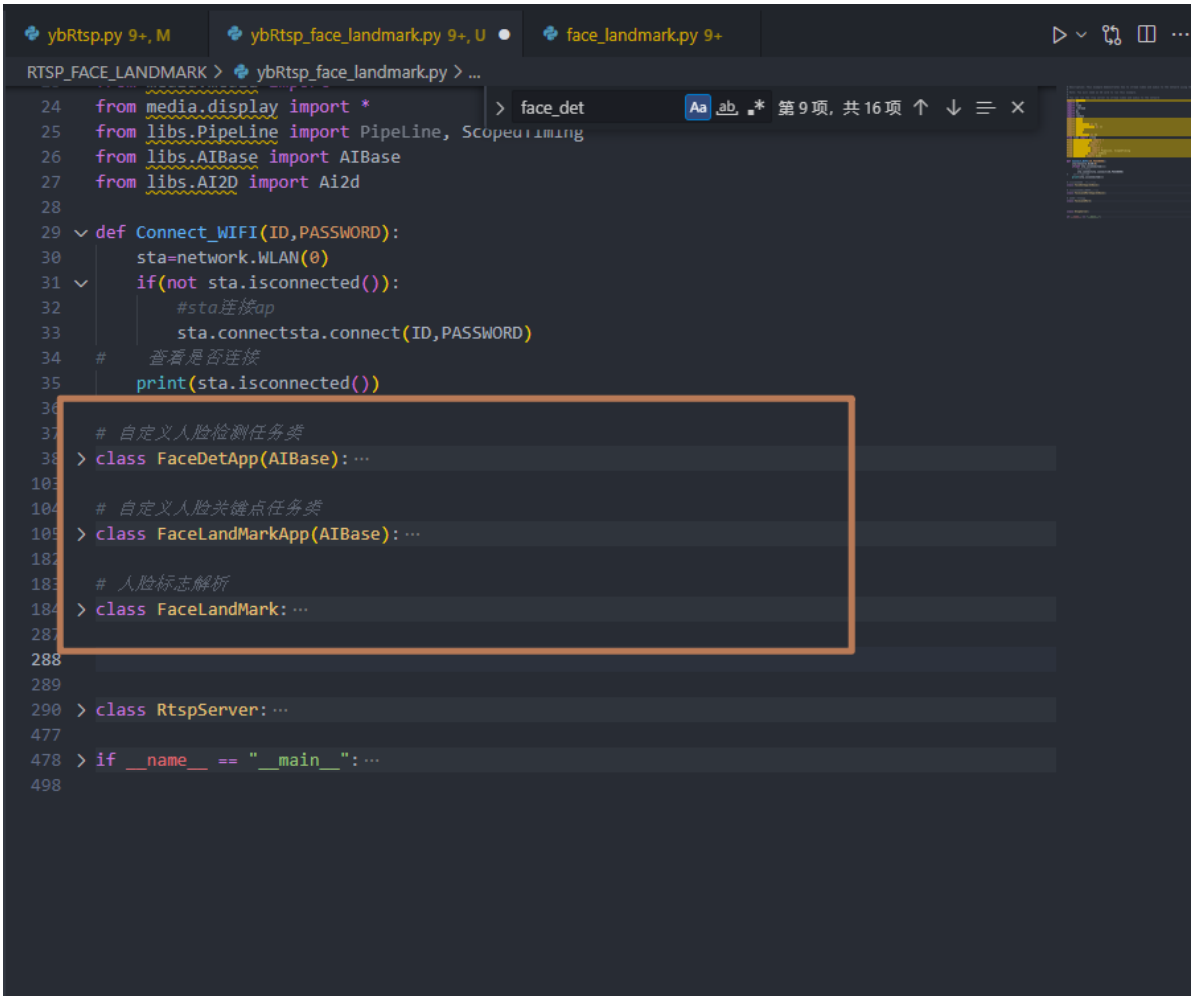
> 由于代码过长所以我用了折叠，这三个类的全部代码我们都要复制过去

```
# 自定义人脸检测任务类
class FaceDetApp(AIBase):...

# 自定义人脸关键点任务类
class FaceLandMarkApp(AIBase):...

# 人脸标志解析
class FaceLandMark:...
```

复制后我们粘贴到【empty_rtsp_demo_face_landmark.py】文件中



我们回到【face_landmark.py】文件里面，找到exce_demo中的while循环部分

现在我们要把这部分移植到RTSP推流的过程中。为了方便观看，我把这部分截图放在边上。

```
try:
    while True:
        img=pl.get_frame()                          # 获取当前帧
        det_boxes,landmark_res=flm.run(img)         # 推理当前帧
        flm.draw_result(pl,det_boxes,landmark_res)  # 绘制推理结果
        pl.show_image()                             # 展示推理效果
        gc.collect()
        time.sleep_us(10)
```

然后我们回到【empty_rtsp_demo_face_landmark.py】中，找到棕色框这一部分（在_do_rtsp_stream()方法里面）

```
try:
    while True:
        img=pl.get_frame()                          # 获取当前帧
        det_boxes,landmark_res=flm.run(img)         # 推理当前帧
        flm.draw_result(pl,det_boxes,landmark_res)  # 绘制推理结果
        pl.show_image()                             # 展示推理效果
        gc.collect()
        time.sleep_us(10)
```

这里是face_landmark.py中的exce_demo部分的截图

```
415     while self.start_stream:
416
417         # 通过两个通道获取图片
418         img = self.sensor.snapshot(chn=CAM_CHN_ID_1)
419         np_img = img.to_numpy_ref()
420         # res = face_det.run(np_img)          # 推理当前帧
421         res = flm.run(np_img)          # 推理当前帧
422         rtsp_show_img = self.sensor.snapshot(chn=CAM_CHN_ID_0)
423
424         # 绘制AI视觉结果
425         # face_det.draw_result(rtsp_show_img,res)
426         flm.draw_result(rtsp_show_img,res)
427          rtsp_show_img = None
428
429          if(rtsp_show_img is None):
430              rtsp_show_img = self.sensor.snapshot(chn=CAM_CHN_ID_0)
431              rtsp_show_img.clear()
```

我们按照截图中的写法，将棕色框中的部分修改，改完后的代码如下：

```
try:
    while True:
        img=pl.get_frame()                          # 获取当前帧
        det_boxes,landmark_res=flm.run(img)         # 推理当前帧
        flm.draw_result(pl,det_boxes,landmark_res)  # 绘制推理结果
        pl.show_image()                             # 展示推理效果
        gc.collect()
        time.sleep_us(10)
```

```
414         # 通道二用来推流显示
415         while self.start_stream:
416
417             # 通过两个通道获取图片
418             img = self.sensor.snapshot(chn=CAM_CHN_ID_1)
419             np_img = img.to_numpy_ref()
420             # res = face_det.run(np_img)          # 推理当前帧
421             det_boxes,landmark_res = flm.run(np_img)          # 推理当
422             rtsp_show_img = self.sensor.snapshot(chn=CAM_CHN_ID_0)
423
424             # 绘制AI视觉结果
425             # face_det.draw_result(rtsp_show_img,res)
426             flm.draw_result(rtsp_show_img,det_boxes,landmark_res)
427 #            rtsp_show_img = None
428
```

> res = flm.run(np_img) 改为 det_boxes,landmark_res = flm.run(np_img)
>
> flm.draw_result(rtsp_show_img, res) 改为
> flm.draw_result(rtsp_show_img,det_boxes,landmark_res)
>
> 注意 **pl** 要改为 **rtsp_show_img**

4. 修改FaceLandMark以及相关类

我们找到【FaceLandMark】类中的【draw_result】方法

```
class FaceLandMark:
    def __init__(self,face_det_kmodel,face_landmark_kmodel,det_input_size,landmar

    # run函数
    def run(self,input_np):...


    # 绘制人脸解析效果
    def draw_result(self,pl,dets,landmark_res):
        pl.osd_img.clear()
        if dets:
            draw_img_np = np.zeros((self.display_size[1],self.display_size[0],4)
            draw_img = image.Image(self.display_size[0], self.display_size[1], ir
            for pred in landmark_res:
                #  (1) 获取单个人脸框对应的人脸关键点
                for sub_part_index in range(len(self.dict_kp_seq)):
                    #  (2) 构建人脸某个区域关键点集
                    sub_part = self.dict_kp_seq[sub_part_index]
                    face_sub_part_point_set = []
                    for kp_index in range(len(sub_part)):
                        real_kp_index = sub_part[kp_index]
                        x, y = pred[real_kp_index * 2], pred[real_kp_index * 2 +
                        x = int(x * self.display_size[0] // self.rgb888p_size[0]
                        y = int(y * self.display_size[1] // self.rgb888p_size[1]
                        face_sub_part_point_set.append((x, y))
                    #  (3) 画人脸不同区域的轮廓
                    if sub_part_index in (9, 6):
                        color = np.array(self.color_list_for_osd_kp[sub_part_ind
                        face_sub_part_point_set = np.array(face_sub_part_point_s
```

1. 修改参数中的pl为img
2. 删除 pl.osd_img.clear() 这一行
3. 找到draw_img.draw_circle()这一行，修改为img.draw_circle()，并删除 pl.osd_img.copy_from(draw_img)这一行

```
259             for pred in landmark_res:
260                 #  (1) 获取单个人脸框对应的人脸关键点
261                 for sub_part_index in range(len(self.dict_kp_seq)):
262                     #  (2) 构建人脸某个区域关键点集
263                     sub_part = self.dict_kp_seq[sub_part_index]
264                     face_sub_part_point_set = []
265                     for kp_index in range(len(sub_part)):
266                         real_kp_index = sub_part[kp_index]
267                         x, y = pred[real_kp_index * 2], pred[real_kp_index * 2 + 1]
268                         x = int(x * self.display_size[0] // self.rgb888p_size[0])
269                         y = int(y * self.display_size[1] // self.rgb888p_size[1])
270                         face_sub_part_point_set.append((x, y))
271                     #  (3) 画人脸不同区域的轮廓
272                     if sub_part_index in (9, 6):
273                         color = np.array(self.color_list_for_osd_kp[sub_part_index],dtype = np.uint8)
274                         face_sub_part_point_set = np.array(face_sub_part_point_set)
275                         aidemo.polylines(draw_img_np, face_sub_part_point_set,False,color,5,8,0)
276                     elif sub_part_index == 4:
277                         color = self.color_list_for_osd_kp[sub_part_index]
278                         for kp in face_sub_part_point_set:
279                             x,y = kp[0],kp[1]
280                             draw_img.draw_circle(x,y ,2, color, 1)
281                     else:
282                         color = np.array(self.color_list_for_osd_kp[sub_part_index],dtype = np.uint8)
283                         face_sub_part_point_set = np.array(face_sub_part_point_set)
284                         aidemo.contours(draw_img_np, face_sub_part_point_set,-1,color,2,8)
285             pl.osd_img.copy_from(draw_img)
286
287
288
```

```
                face_sub_part_point_set.append((x, y))
            # （3）画人脸不同区域的轮廓
            if sub_part_index in (9, 6):
                color = np.array(self.color_list_for_osd_kp[sub_part_index],dtype = np.uint8)
                face_sub_part_point_set = np.array(face_sub_part_point_set)
                aidemo.polylines(draw_img_np, face_sub_part_point_set,False,color,5,8,0)
            elif sub_part_index == 4:
                color = self.color_list_for_osd_kp[sub_part_index]
                for kp in face_sub_part_point_set:
                    x,y = kp[0],kp[1]
                    img.draw_circle(x,y ,2, color, 1)
            else:
                color = np.array(self.color_list_for_osd_kp[sub_part_index],dtype = np.uint8)
                face_sub_part_point_set = np.array(face_sub_part_point_set)
                aidemo.contours(draw_img_np, face_sub_part_point_set,-1,color,2,8)
    # pl.osd_img.copy_from(draw_img)
```

修改后

> 此时需要分情况讨论
>
> 1. 如果例程绘制的线条很少（比如说人脸检测或者人脸识别），那么大概率图像是直接绘制在img上面的【关键特征是出现img.draw_xxxx()这样的方法】
>
>   这种情况下改到这里就可以了
>
> 2. 如果例程绘制的线条很多（比如人脸关键点绘制之类的），除了直接的draw_Xxxx()还可能有【aidemo.xxxx()】这些绘制函数。在这种情况下，需要我们灵活的修改代码。思路是直接获取到识别的关键点，然后手动添加在图像上绘制的方法

4. 本例程中绘制的是复杂图像，其中眼睛部分原代码使用了draw_circle方法，我们可以直接按照步骤三【找到draw_img.draw_circle()这一行，修改为img.draw_circle()】。但是其它关键点处使用了aidemo.polylines() 和 aidemo.contours() 方法，这两个方法的参数必须是rgb888类型的图片转为的nparray，和我们传进来的图片参数img不符（img是Yuv420sp格式的，RTSP实时图传只能使用这种格式的图片），所以我们需要自行添加绘制函数。在本次例程中，我们在获取到关键点后，用draw_circle方法手动绘制这些关键点

```
class FaceLandMark:
    def draw_result(self,img,dets,landmark_res):
        for pred in landmark_res:
            # （1）获取单个人脸框对应的人脸关键点
            for sub_part_index in range(len(self.dict_kp_seq)):
                # （2）构建人脸某个区域关键点集
                sub_part = self.dict_kp_seq[sub_part_index]
                face_sub_part_point_set = []
                for kp_index in range(len(sub_part)):
                    real_kp_index = sub_part[kp_index]
                    x, y = pred[real_kp_index * 2], pred[real_kp_index * 2 + 1]
                    x = int(x * self.display_size[0] // self.rgb888p_size[0])
                    y = int(y * self.display_size[1] // self.rgb888p_size[1])
                    # 手动添加绘制方法 ################################
                    img.draw_circle(x,y ,1, (255,0,0), 1)
                    ##################################################
                    face_sub_part_point_set.append((x, y))
                # （3）画人脸不同区域的轮廓
                if sub_part_index in (9, 6):
                    color = np.array(self.color_list_for_osd_kp[sub_part_index],dtype
                    face_sub_part_point_set = np.array(face_sub_part_point_set)
                    aidemo.polylines(draw_img_np, face_sub_part_point_set,False,color,
                elif sub_part_index == 4:
                    color = self.color_list_for_osd_kp[sub_part_index]
                    for kp in face_sub_part_point_set:
```

注意事项【必看】

Q: 为什么我K230 CanMV IDE显示[RTSP]启动成功了，也有地址信息，但是视频就是连不上？

1. 如果地址是0.0.0.0，那么大概率是网络连接出问题了，请检查是否能正确连接到WIFI

2. 请确保想要远程观看的电脑或手机和K230连接的是同一个WIFI （或在同一局域网内）
3. 经测试，只能有小于等于两台设备同时连接，超过这个数量的后面的就连不上了
4. 排除了以上的情况后，若依然无法连接RTSP，请把K230断开与电源的链接并按下RST复位键，等待10秒后再重新上电运行程序

Q: 为什么我绘制的颜色与我设定的不符？

A: 因为目前img绘制的方法在推流使用的图片格式（Yuv420sp）上兼容性不好，传入RGB格式的颜色不能被正常的解析，这个问题目前暂时没有很好的解决方案

Q: 为什么我绘制一个矩形框，图片上出现了多个？

A: 请确保display_size设置的是 [1280x720]

Q: 我在使用手机版的VLC for android播放器观看时存在很高的延迟？

A: 这是播放器的设置决定的，这个app我们没有找到相关设置的地方